

Computer Networks

Lecture 5

Shmulik Kaplan

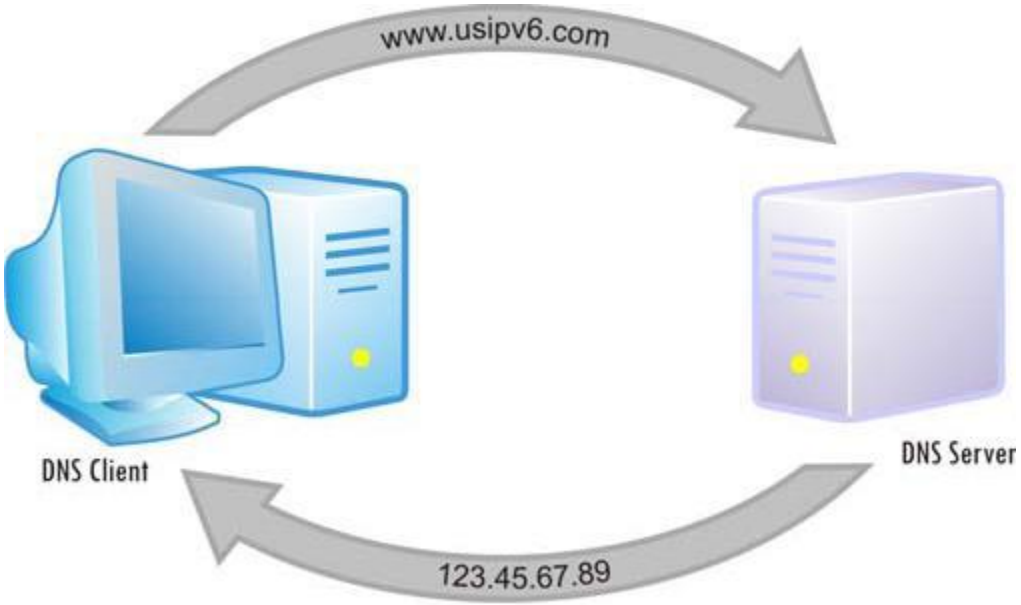
Today

- DNS:
  - General description & caching
  - Actual messages
  - DNS lookup
  - DNS privacy
  - CDN
- Consensus Algorithms introduction
- Consensus Algorithm – Crash failures
- Consensus Algorithm – Byzantine failures

שרות תרגום שמות

DNS: Domain Name System

The internet's directory service – translates hostnames to IP addresses



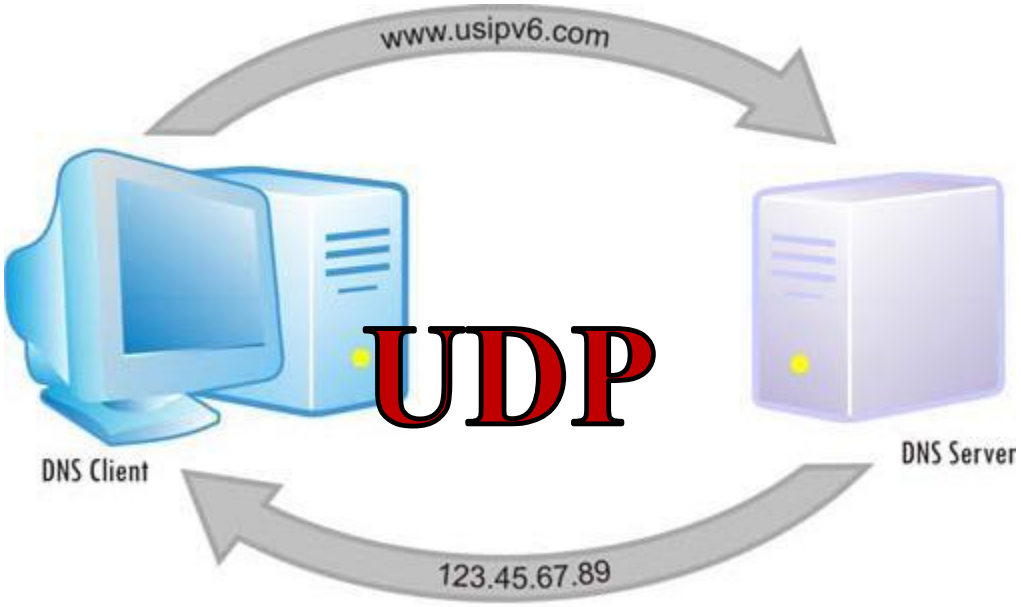
Domain Name System

- *distributed database* implemented in hierarchy of many *name servers*
- *application-layer protocol* -- communicate to *resolve* names

שרות תרגום שמות

DNS: Domain Name System

The internet's directory service – translates hostnames to IP addresses



Domain Name System

- *distributed database* implemented in hierarchy of many *name servers*
- *application-layer protocol* -- communicate to *resolve* names

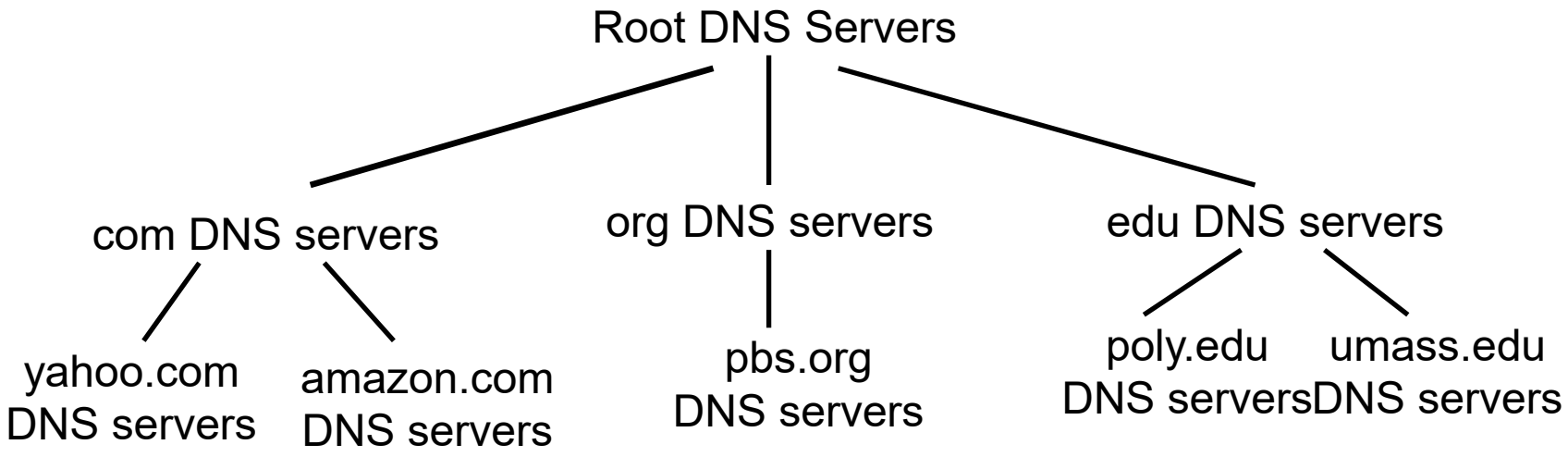


Why not centralize DNS?

- single point of failure
- traffic volume
- distant centralized database
- maintenance

doesn't *scale!*

DNS: a distributed, hierarchical database



Root DNS Servers

Top-level domain (TLD) servers

Authoritative DNS servers

Local DNS server

DNS: root name servers

13 root name "servers" worldwide



TLD, authoritative servers

top-level domain (TLD) servers:

- responsible for com, org, net, edu, aero, jobs, museums, and all top-level country domains, e.g.: uk, fr, ca, jp
- Network Solutions maintains servers for .com TLD
- Educause for .edu TLD

authoritative DNS servers:

- Organizations own DNS server(s), providing authoritative hostname to IP mappings for organization's named hosts
- can be maintained by organization or service provider

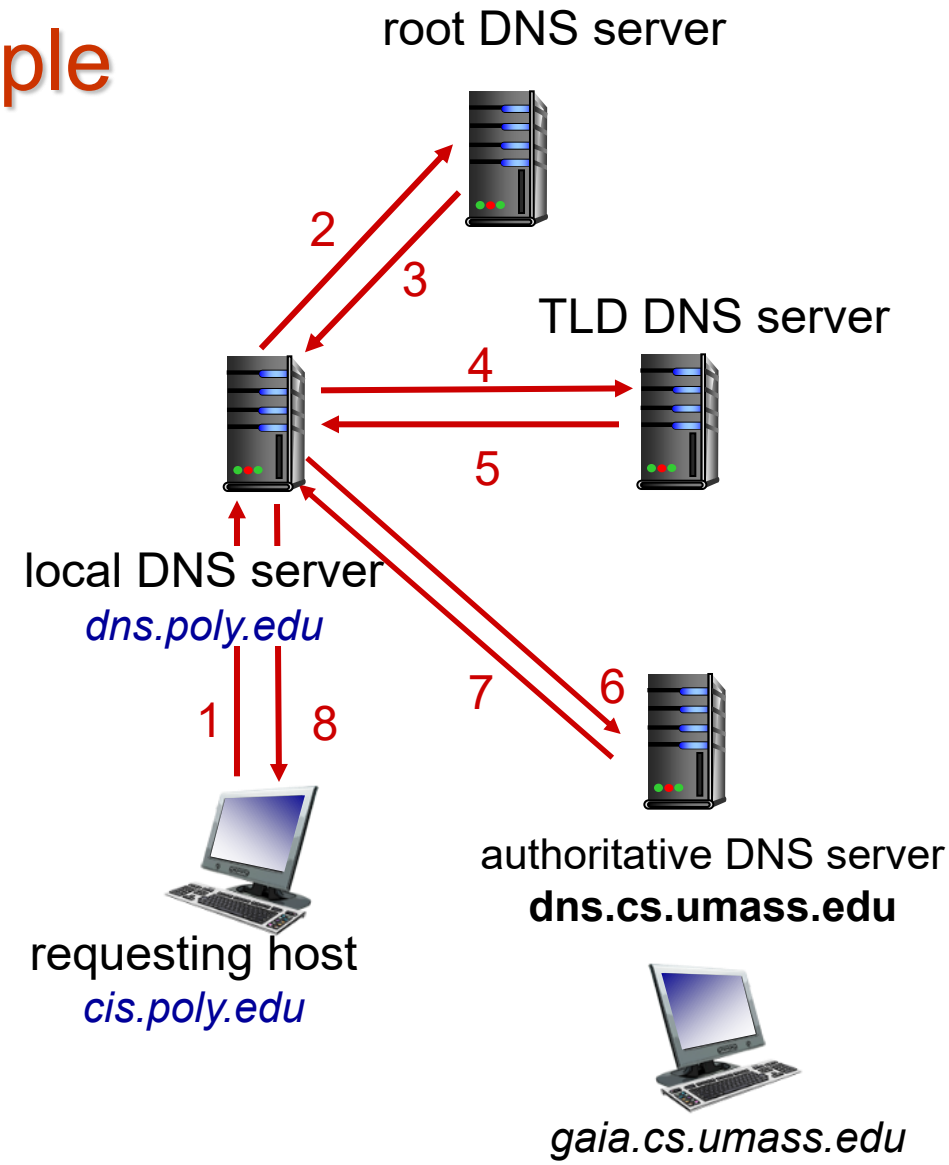
Local DNS name server

- does not strictly belong to hierarchy
- each ISP (residential ISP, company, university) has one
  - also called "default name server"
- when host makes DNS query, query is sent to its local DNS server
  - has local cache of recent name-to-address translation pairs (but may be out of date!)
  - acts as proxy, forwards query into hierarchy

DNS name resolution example

iterated query:

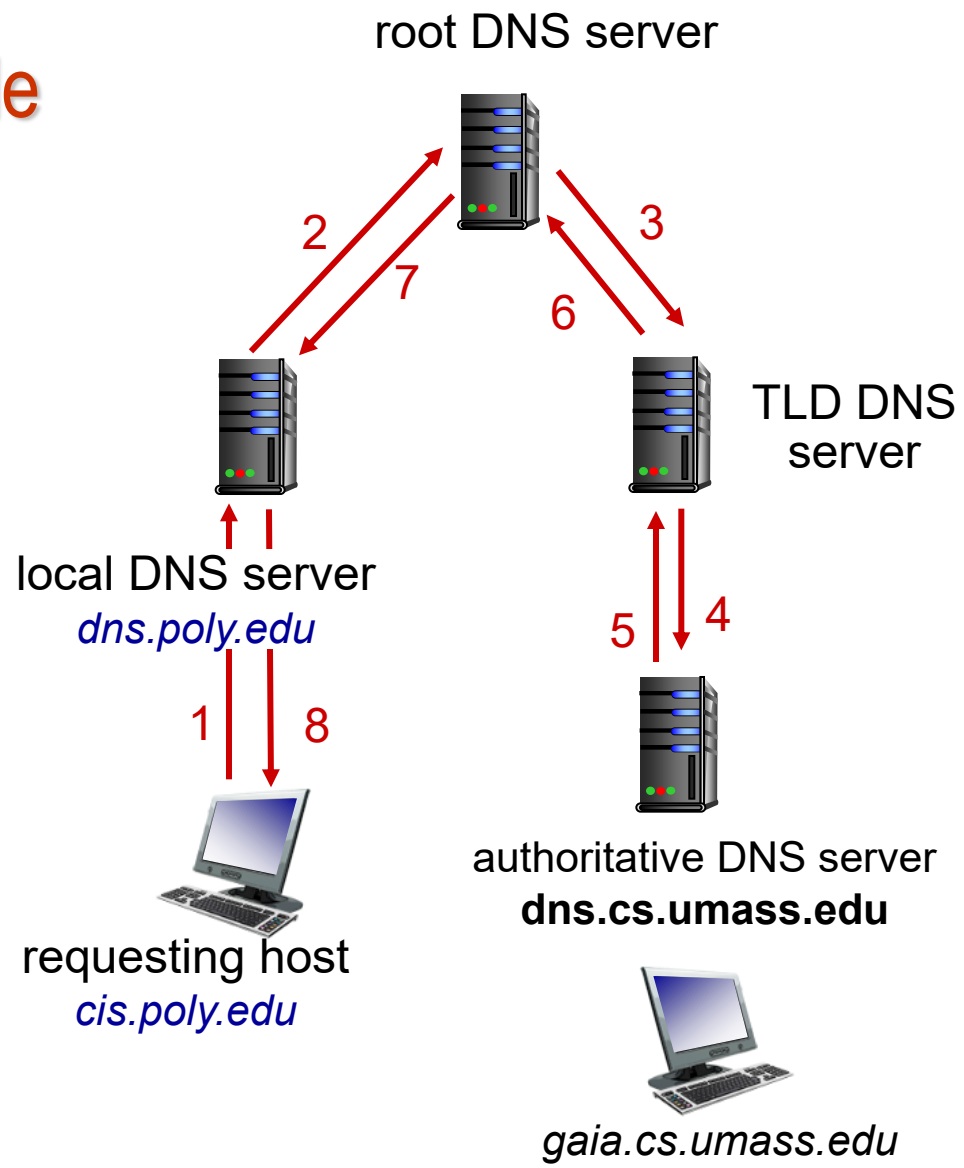
- contacted server replies with name of server to contact
- "I don't know this name, but ask this server"



DNS name resolution example

recursive query:

- puts burden of name resolution on contacted name server
- heavy load at upper levels of hierarchy?



DNS: caching

- Once (any) name server learns mapping, it *caches* mapping
  - cache entries timeout (disappear) after some time
  - TLD servers typically cached in local name servers. Thus, root name servers not often visited



Today

- ✓ DNS:
  - ✓ General description & caching
  - Actual messages
  - DNS lookup
  - DNS privacy
  - CDN
- Consensus Algorithms introduction
- Consensus Algorithm – Crash failures
- Consensus Algorithm – Byzantine failures

DNS records

DNS: distributed db storing Resource Records (RR)

RR format: **`(name, value, type,TTL)`**

Type=A

- **`name`** is hostname
- **`value`** is IP address
- **(www.foo.com, 145.37.93.126, A)**

Type=CNAME

- **`name`** is an alias name for some "cannonical" (the real) name
- **`value`** is cannonical name
- **(foo.com, relay1.foo.com, CNAME)**

Type=NS

- **`name`** is domain (e.g., foo.com)
- **`value`** is hostname of authoritative name server for this domain
- **(foo.com, dns.foo.com, NS)**

Type=MX

- **`value`** is hostname of mailserver associated with **`name`**
- **(foo.com, mail.bar.foo.com, MX)**

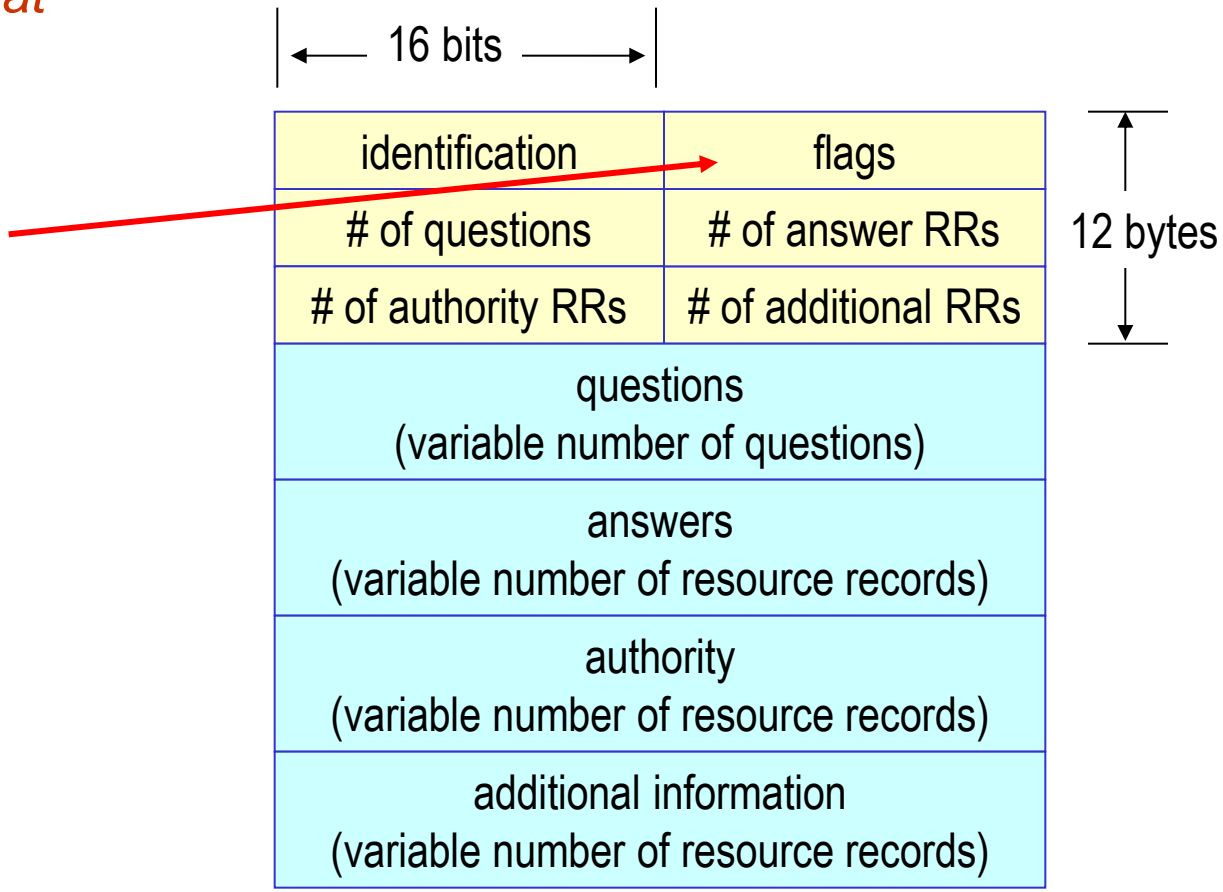
- TTL = time to live (determine the time to remove from a cache)

DNS protocol, messages

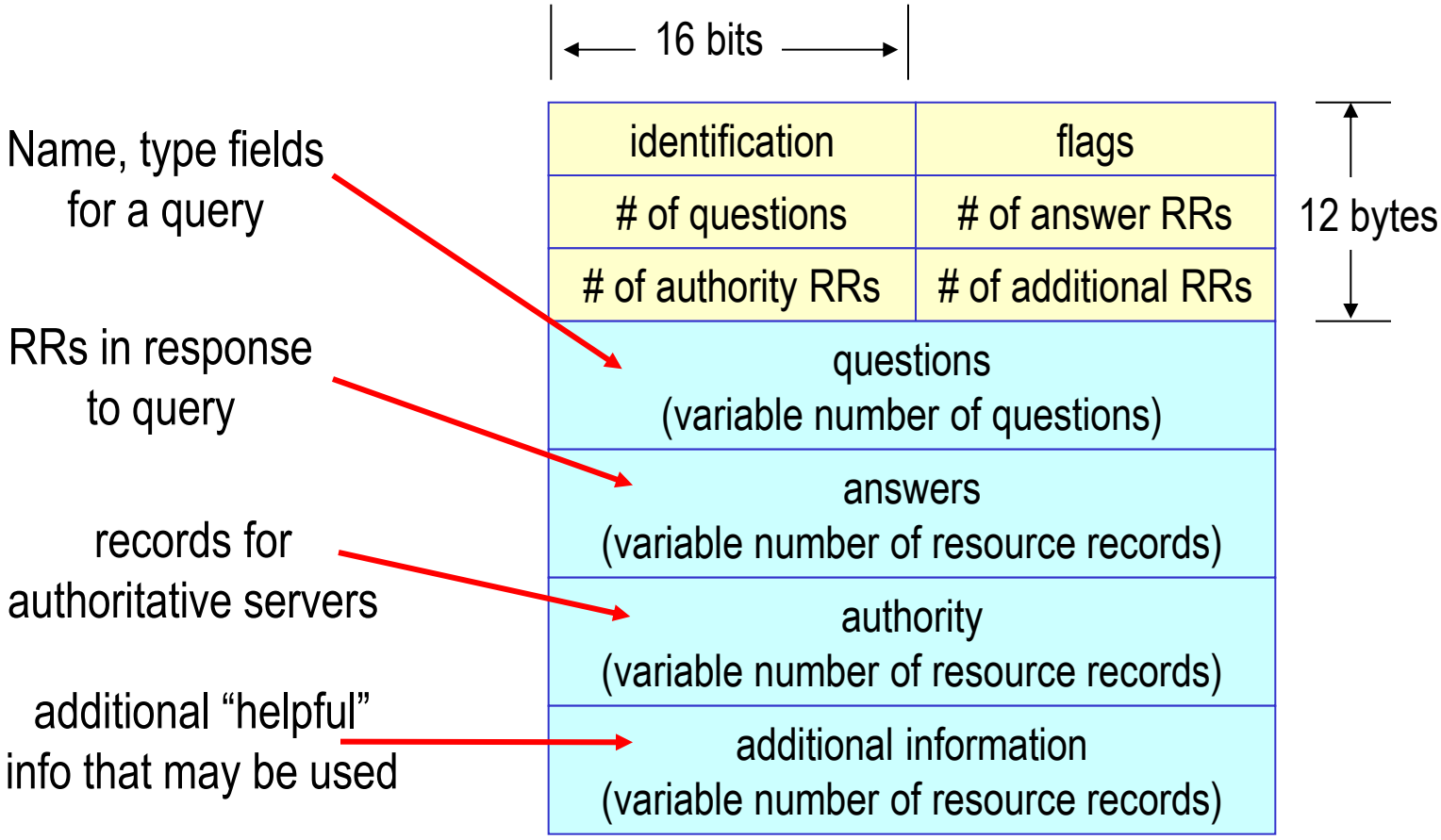
DNS protocol : *query* and *reply* messages, both with same *message format*

Flags:

- query or reply
- recursion desired
- recursion available
- reply is authoritative



DNS protocol, messages



Inserting records into DNS

- Example: just created startup “Network Utopia”
- Register name networkuptopia.com at a registrar (e.g., Network Solutions)
  - Need to provide registrar with names and IP addresses of your authoritative name server (primary and secondary)
  - Registrar inserts two RRs into the com TLD server:

```
(networkutopia.com, dns1.networkutopia.com, NS)
(dns1.networkutopia.com, 212.212.212.1, A)
```

- Put in authoritative server Type A record for www.networkuptopia.com and Type MX record for networkutopia.com

networkutopia.com

Today

- ✓ DNS:
  - ✓ General description & caching
  - ✓ Actual messages
  - DNS lookup
  - DNS privacy
  - CDN
- Consensus Algorithms introduction
- Consensus Algorithm – Crash failures
- Consensus Algorithm – Byzantine failures

DNS lookup - nslookup

```
Select C:\Windows\system32\cmd.exe
Microsoft Windows [Version 10.0.19045.2251]
(c) Microsoft Corporation. All rights reserved.

C:\Users\User>nslookup www.runi.ac.il
Server:  UnKnown
Address:  2a06:c701:436c:5200:2b8:c2ff:fea1:1245

Non-authoritative answer:
Name:    www.runi.ac.il
Address:  194.153.101.3


C:\Users\User>nslookup www.runi.ac.il 8.8.8.8
Server:  dns.google
Address:  8.8.8.8

Non-authoritative answer:
Name:    www.runi.ac.il
Address:  194.153.101.3

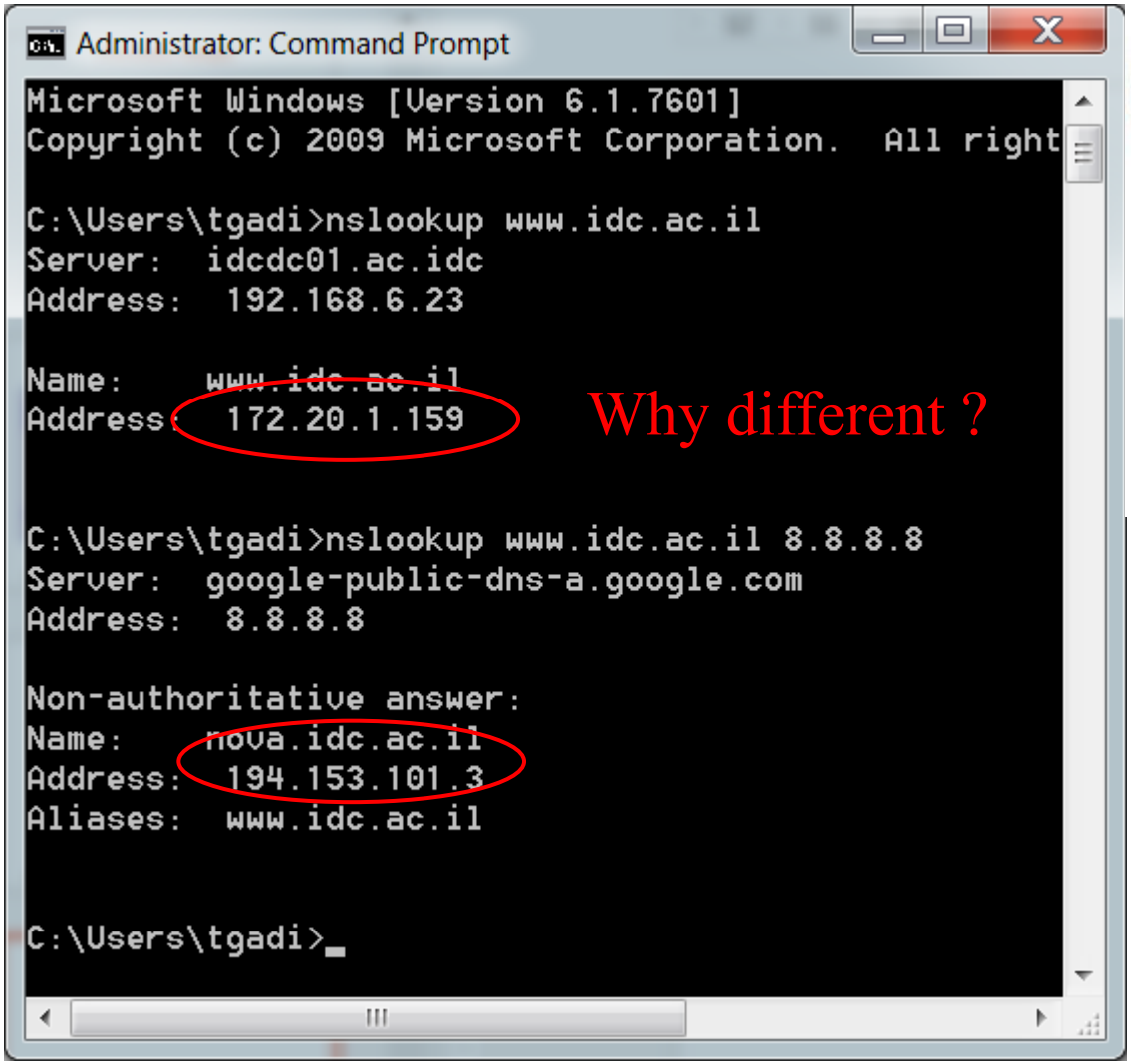

C:\Users\User>
```





stay safe online with 9.9.9.9

DNS lookup split view



Why different ?







Today

- ✓ DNS:
  - ✓ General description & caching
  - ✓ Actual messages
  - ✓ DNS lookup
  - DNS privacy
  - CDN
- Consensus Algorithms introduction
- Consensus Algorithm – Crash failures
- Consensus Algorithm – Byzantine failures

DNS Privacy
Encryption between DNS client and DNS resolver

An IETF study in July 2019 found that:

- July 2019: 8.5% of the Networks intercept DNS queries (27.9% in China)
- (today) most queries are answered un-altered, but the situation might change, intercept server might change DNS answers.
- SOLUTIONs?
  - DNS over TLS (DoT). RFC 7858 (May 2016); RFC 8310 (March 2018)
  - DNS over HTTPS (DoH). RFC 8484 (Octobe



DoT vs DoH

Category	**DoT (DNS over TLS)**	**DoH (DNS over HTTPS)**
How it works	Encrypts DNS directly using TLS on port **853**	Sends DNS inside **HTTPS (HTTP/2/3)** on port **443**
Traffic appearance	Looks like encrypted DNS	Looks like regular HTTPS web traffic
Blocking / Filtering	Easy to block (dedicated port)	Hard to block; indistinguishable from normal HTTPS
Performance	Lower overhead (simple TLS)	Higher overhead (HTTP + TLS + headers)
Network compatibility	May be blocked on restrictive networks	Works even on very restricted networks (hotspots, corporate)
Control / Monitoring	Easier for enterprises to inspect/log	Harder; can bypass OS and enterprise DNS policies
Privacy benefits	Encrypts DNS, but visible as DNS	Encrypts DNS and hides it inside web traffic

DNS Privacy
DNS over TLS (DoT) and DNS over HTTPS (DoH)



Differences between DoT and DoH

- DoT can be easily blocked, because it is running on a dedicated port (853)
- DoH is made to look like normal HTTPS traffic (port 443), selective blocking of DoH is difficult.
- DoH enables developers to do DNS name resolutions as part of their application (Chrome, Firefox), which some people think is very bad.
- Adopted by Firefox and Chrome (but not as default).

Interesting talk about this topic (September 2019):
https://www.youtube.com/watch?v=pjin3nv8jAo
(~ 40 minutes)

Today

- ✓ DNS:
  - ✓ General description & caching
  - ✓ Actual messages
  - ✓ DNS lookup
  - ✓ DNS privacy
  - CDN
- Consensus Algorithms introduction
- Consensus Algorithm – Crash failures
- Consensus Algorithm – Byzantine failures

Content Distribution Networks (CDNs)



Content Distribution Networks (CDNs)

- Video traffic: major consumer of Internet bandwidth

- Challenge: scale - how to reach ~1B users?







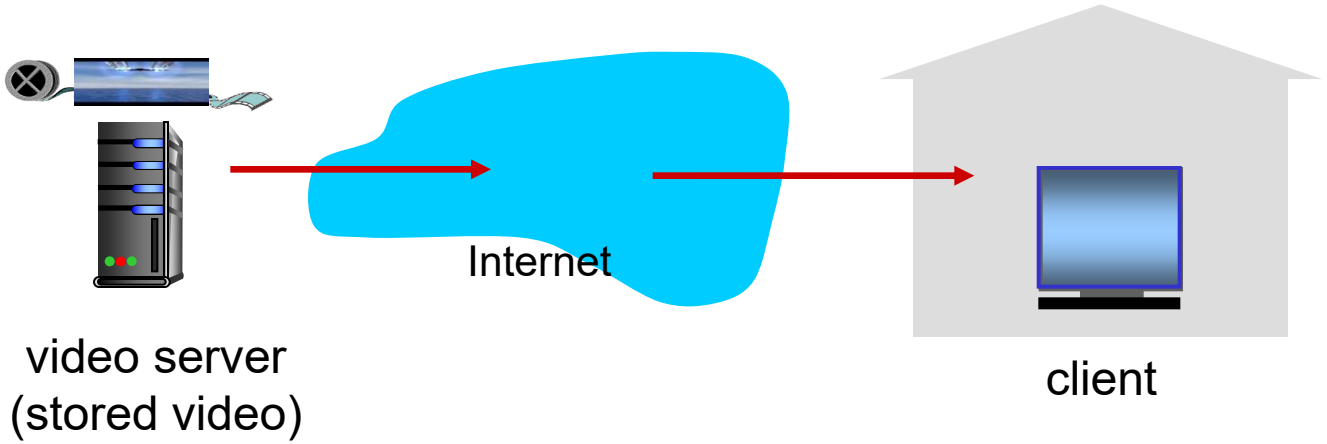






Streaming stored video

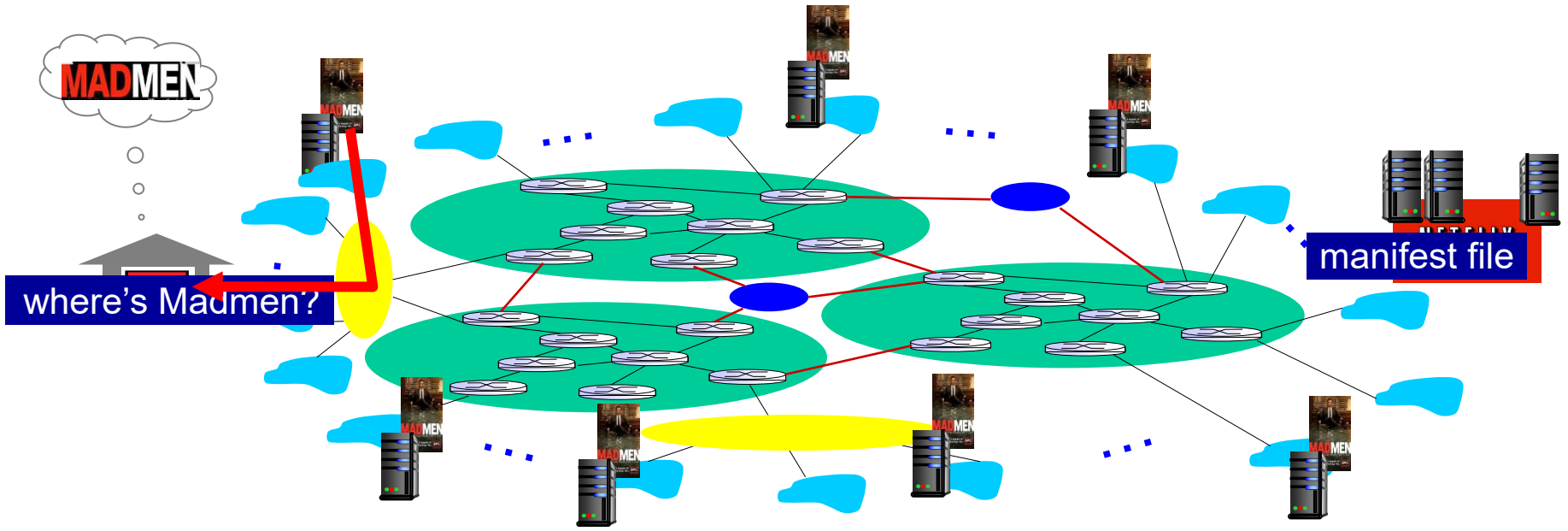
simple scenario:



Single mega-video server won't work
(why?)

Content Distribution Networks (CDNs)

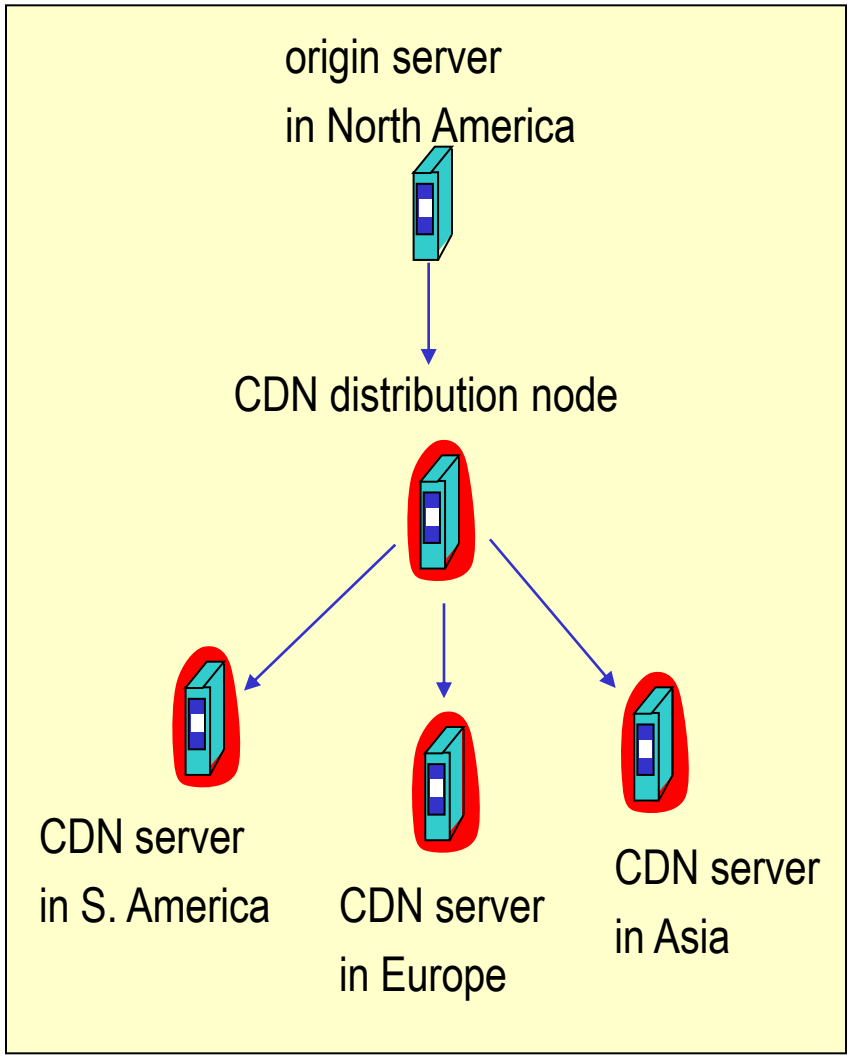
CDN: stores copies of content at CDN nodes
e.g. Netflix stores copies of MadMen



Content Distribution Networks (CDNs)

Content replication

- CDN company installs hundreds of CDN servers throughout Internet (in lower-tier ISPs, close to users)
- CDN replicates its customers' content in CDN servers. When provider updates content, CDN updates servers
  - replicate all content
  - replicate partial content
- The content providers are the CDN customers
- Make use of DNS name resolution!



Content Distribution Network

הסיפור של חברת אקמאי





Daniel M. Lewin
1970-2001
Akamai Co-Founder and
Chief Technology Officer

"Akamai's mission is to transform the delivery of Internet content so that Web congestion is a thing of the past."

Today

- ✓ DNS:
  - ✓ General description & caching
  - ✓ Actual messages
  - ✓ DNS lookup
  - ✓ DNS privacy
  - ✓ CDN
- Consensus Algorithms introduction
- Consensus Algorithm – Crash failures
- Consensus Algorithm – Byzantine failures

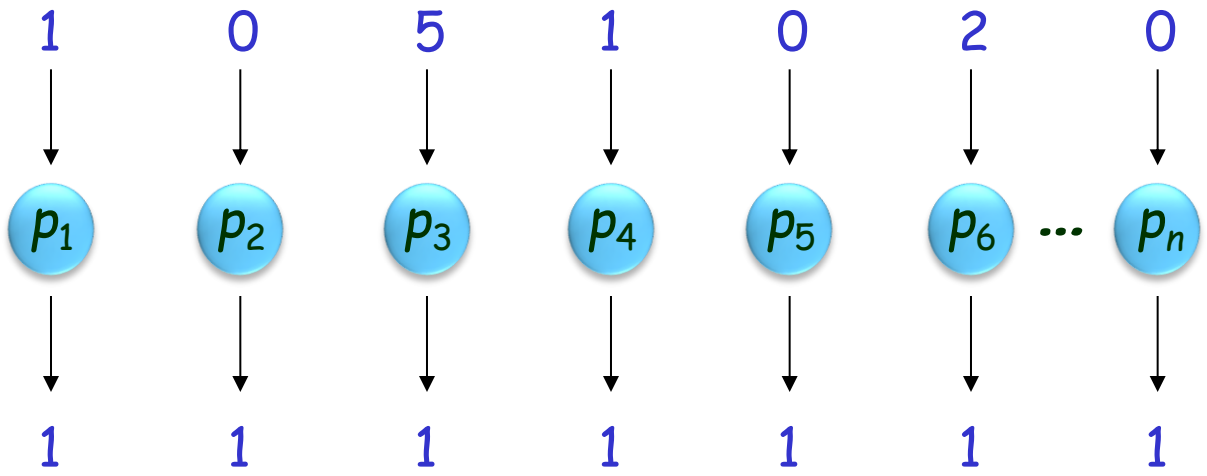
Consensus in Synchronous Networks



Motivation

- In distributed systems, things fail. This means that it is never safe to rely on one copy of anything …
- Google achieves fault-tolerance of their databases through a replication system (which consists of multiple replicas …)
- Microsoft has implemented a storage service that runs on the desktop computers of a large organization and provides the semantics of a central file server.
- Several sensor nodes monitoring the same environment…
- Voting, electing a leader.
- Agreement is essential for the implementation of *blockchain* which is a cryptography technology for implementing a distributed ledger. (Bitcoin for example).

Consensus
Problem definition

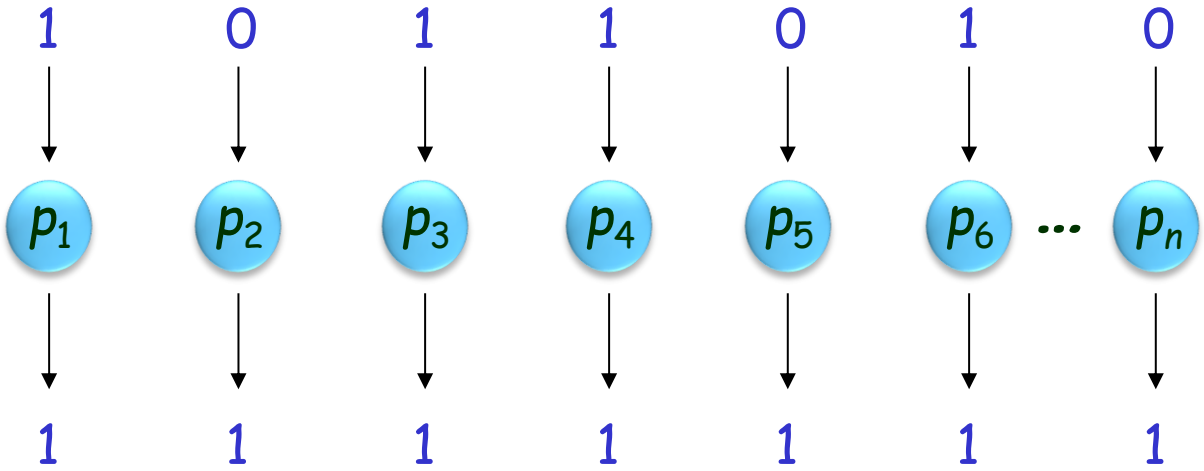


Each process p_i has input value in_i

- agreement: All non-faulty processes eventually decides on the same value *v.*
- validity:
  - *Crash failures: v* must be the input of some process
  - *Byzantine failures: v* must be the input of some non-faulty process

➢ (binary consensus ; multi-valued consensus)

Consensus
(binary consensus)



There is a trivial solution when there are no faults.

What happens when there are faults?

Impossibility of Consensus with One Faulty Process

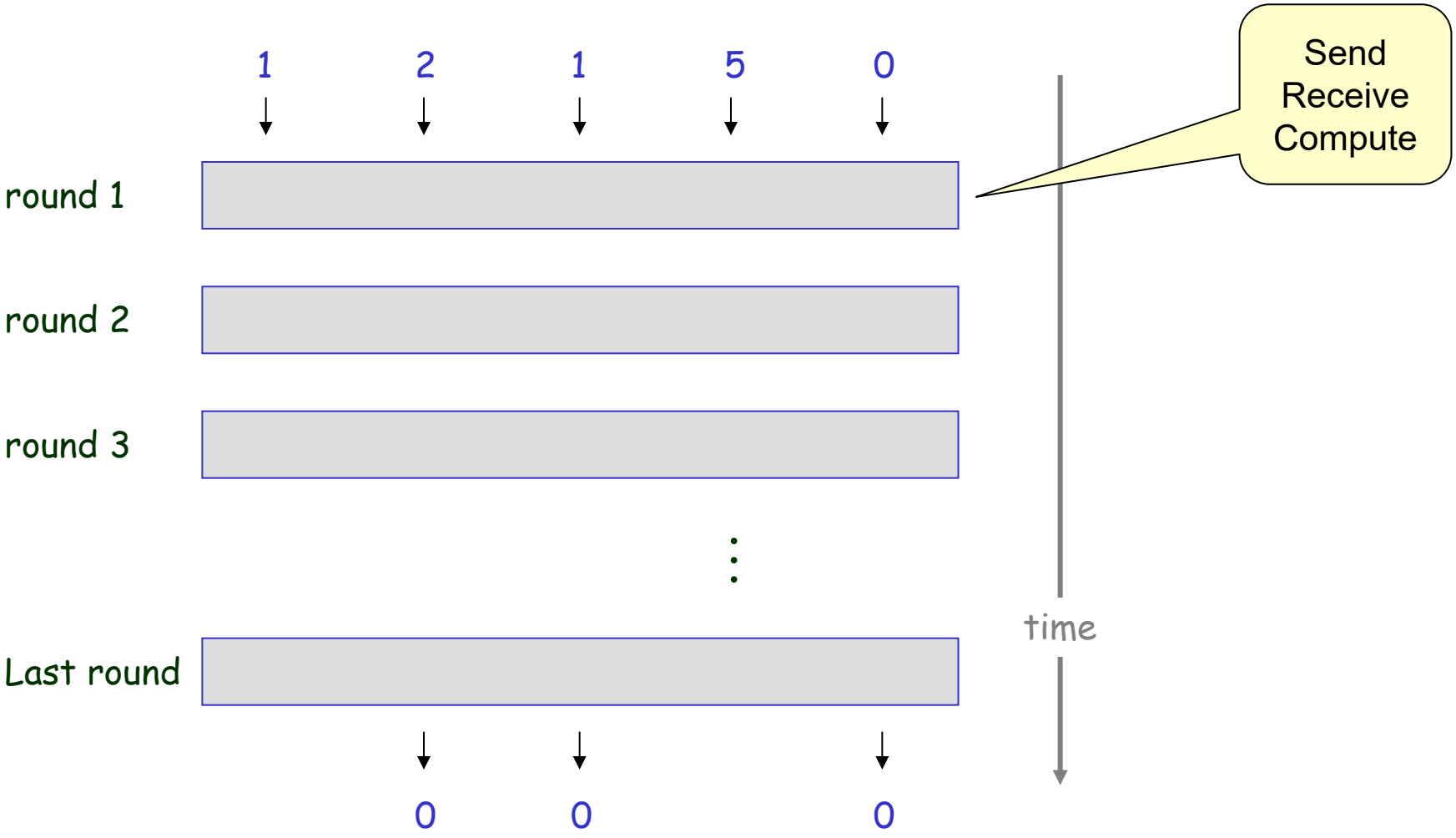
Theorem: There is no asynchronous algorithm that solves the consensus problem using send/receive messages in the presence of a single faulty process.



What can be done?

- Strong primitives.
- Timing assumptions ➔ synchronous model.
- Randomization.

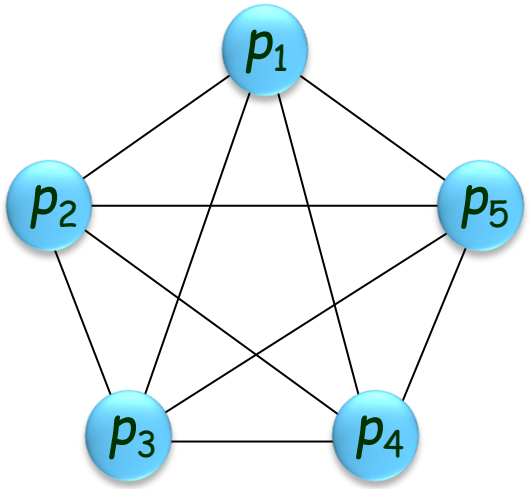
Synchronous Model



Model

Model:

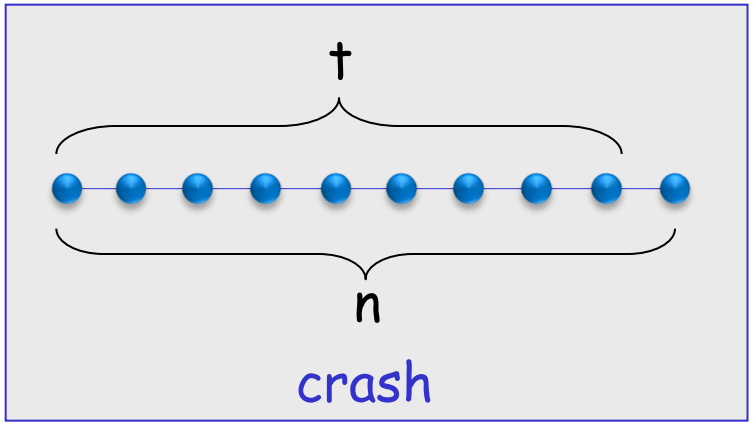
- Synchronous network
- # of processes n is a priori known
- Unique identifiers: 1,…,n
- Network: every process is connected to every other process (clique)
- Communication: message passing
- An upper bound on the # of faulty processes, denoted t, is a priori known
- No failures of links or messages
- Startup condition: all at the same time

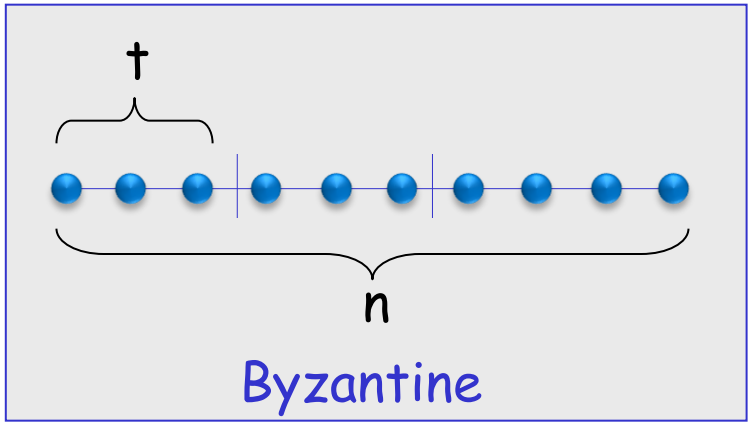


Known tight Bounds

	crash failures	Byzantine failures
number of processes	$n \geq t+1$	$n \geq 3t+1$
number of rounds	$t+1$	$t+1$

t is the maximum number of faulty processes

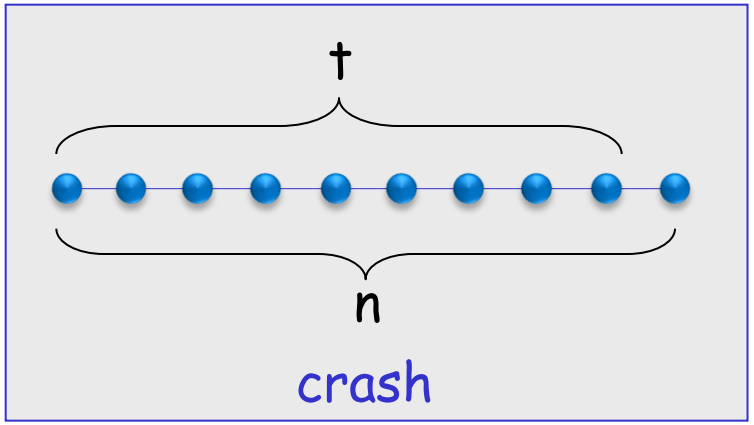


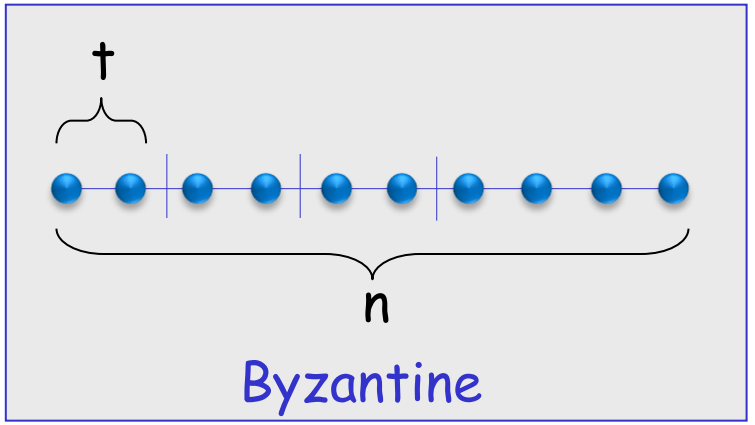


We present two algorithms

	crash failures	Byzantine failures
number of processes	$n \geq t+1$	$n \geq 4t+1$
number of rounds	$t+1$	$2(t+1)$

t is the maximum number of faulty processes





Today

- ✓ DNS:
  - ✓ General description & caching
  - ✓ Actual messages
  - ✓ DNS lookup
  - ✓ DNS privacy
  - ✓ CDN
- ✓ Consensus Algorithms introduction
- Consensus Algorithm – Crash failures
- Consensus Algorithm – Byzantine failures

Consensus for Crash Failures where n ≥ t+1

Algorithm for process p_i with input in_i

```
repeat at each round
   round := round + 1
   if flag = true then send v to all processes
   wait to receive all messages for this round
   temp := v
   v := minimum among all received values at this
        round and current value of v
   if v < temp then flag := true else flag := false
until round = t+1
decide on v
```

Initially:

v	in_i
flag	true
round	0
temp	

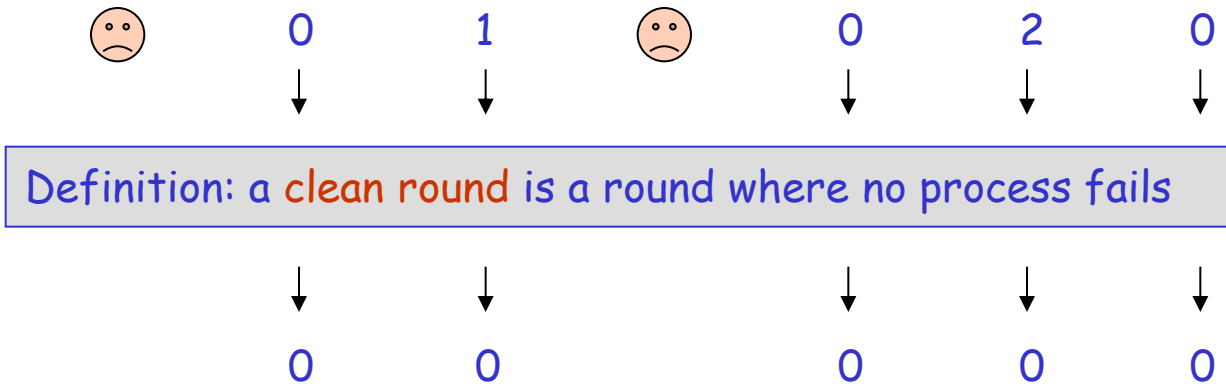
t – the maximum number of faulty processes – is known

Consensus for Crash Failures where n ≥ t+1



Consensus for Crash Failures where n ≥ t+1

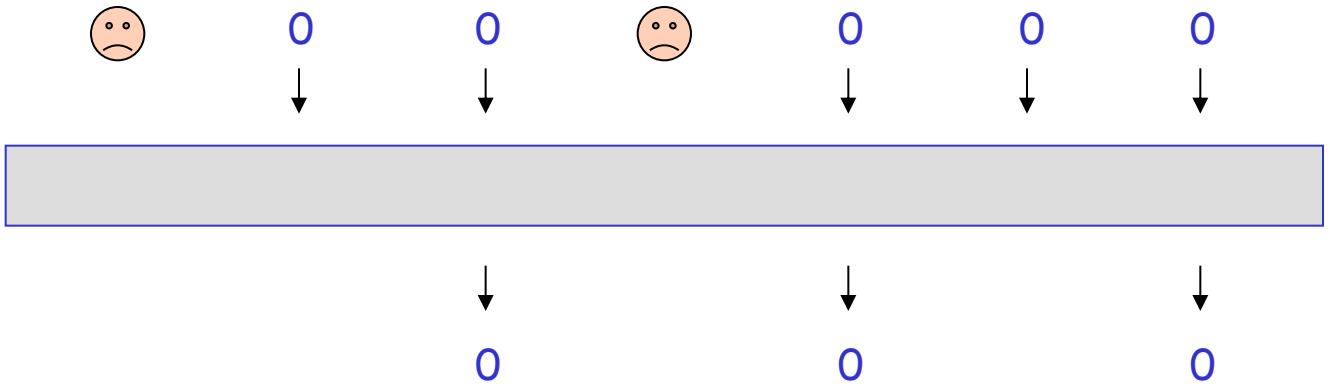
Lemma 1: clean round ➔ same output



time

Consensus for Crash Failures where n ≥ t+1

Lemma 2: same input ➔ same output



time

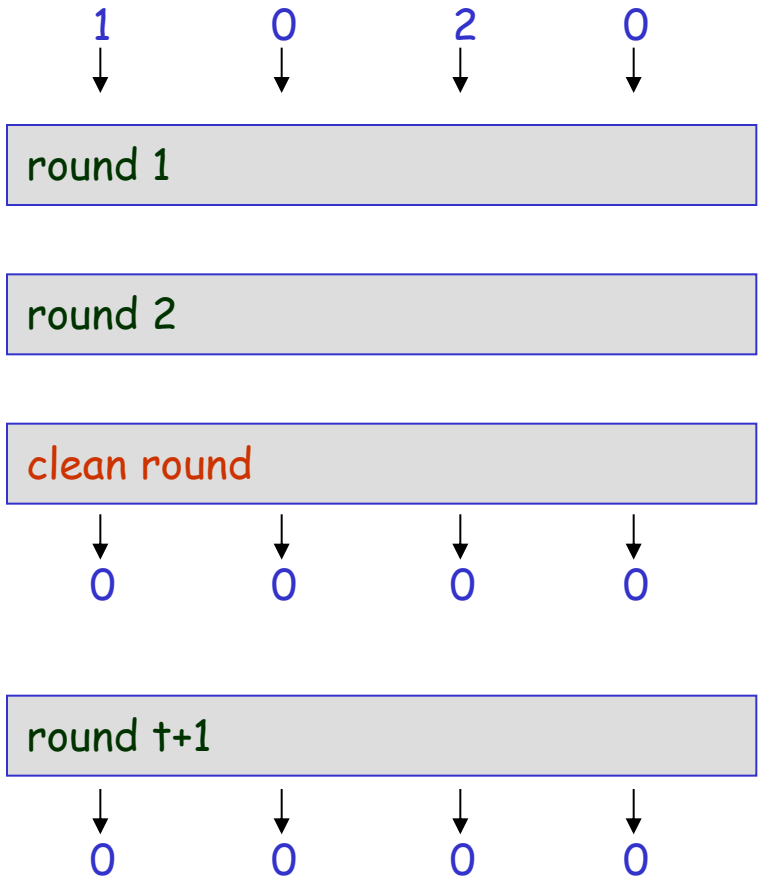
Consensus for Crash Failures where n ≥ t+1

Lemma 1: clean round ➔ same output

Lemma 2: same input ➔ same output

Thm 1: Agreement ✓

Thm 2: Validity ✓



Consensus for Crash Failures where $n \geq t+1$

Summary:

- number of processes: $n \geq t+1$
- number of rounds: $t+1$
- number of messages: $n^2 \bullet \min(t+1, |V|)$
  where V is the input set
- size of each message: $\log|V|$ bits

Question

Is the following algorithm correct ?

repeat at each round

round := round + 1

~~if flag = true then send v to all processes~~

wait to receive all messages for this round

~~temp := v~~

v := minimum among all received values at this round and current value of v

~~if v < temp then flag := true else flag := false~~

until round = t+1

decide on v

Initially:

v in_i

flag true

round 0

temp

Today

- ✓ DNS:
  - ✓ General description & caching
  - ✓ Actual messages
  - ✓ DNS lookup
  - ✓ DNS privacy
  - ✓ CDN
- ✓ Consensus Algorithms introduction
- ✓ Consensus Algorithm – Crash failures
- Consensus Algorithm – Byzantine failures

Binary Consensus for Byzantine Failures where $n \geq 4t+1$

	Byzantine failures
number of processes	$n \geq 4t+1$
number of rounds	$2(t+1)$

The notes cover only the second algorithm.

Presentation of this algorithm is based on:

A Simple proof of a simple consensus algorithm
by J. Misra
Information Processing Letters, (1989) 33:1, 21-24.

Asymptotical optimal Distributed consensus
by P. Berman and J.A. Garay
Proc. ICALP 89. In: LNCS, 372, 80—94, 1989.

Binary Consensus for Byzantine Failures where n ≥ 4t+1

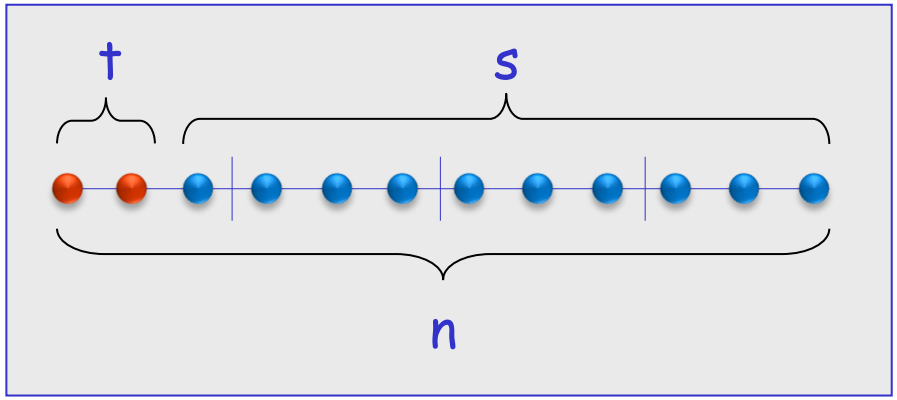
Model:

- Synchronous network
- # of processes n is a priori known (for simplicity assume n is odd)
- Unique identifiers: 1,…,n
- Input is \in {0,1}
- Network: every process is connected to every other process (clique)
- Communication: message passing
- At most t Byzantine failures, where n ≥ 4t+1
- # of failures t is a priori known
- No failures of links or messages
- Startup condition: all at the same time

Binary Consensus for Byzantine Failures where $n \geq 4t+1$

Notations:

- s – correct processes; $s > 3n/4$ (a priori known)
- t – faulty processes; $t < n/4$ (a priori known)
- $n = s + t$ (for simplicity assume that n is odd)



Binary consensus for Byzantine Failures where n ≥ 4t+1

Algorithm for process p_i with input in_i

Processes communicate in rounds. Each round k (k>o), has two sub-rounds.

The algorithm:

v in_i

v = in_i

for k=1 to t+1 do

1. send v to all the processes, and receive messages from all the processes;
2. if my-id = k then v = "majority value from step 1"; send v to all other processes

   else receive a value from process k;

   if in step 1 you have received s or more messages with the same value (including your own) then assign this value to v

   else assign to v the value you have received in step 2.

end loop

decide on v

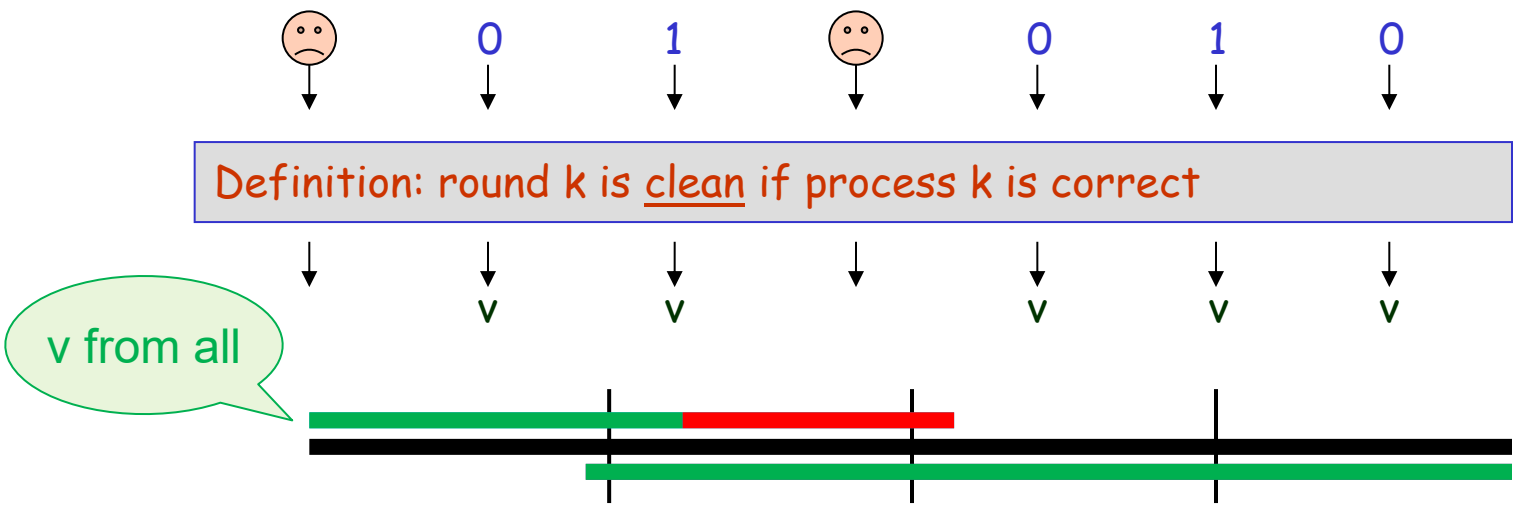
Binary Consensus for Byzantine Failures where n ≥ 4t+1



Binary Consensus for Byzantine Failures where n ≥ 4t+1

Notation: v is the majority value of process k in round k

Lemma 1: round k is clean ➔ same output v



➔ process k got v from > n/2 processes

➔ process k got v from > n/4 correct processes

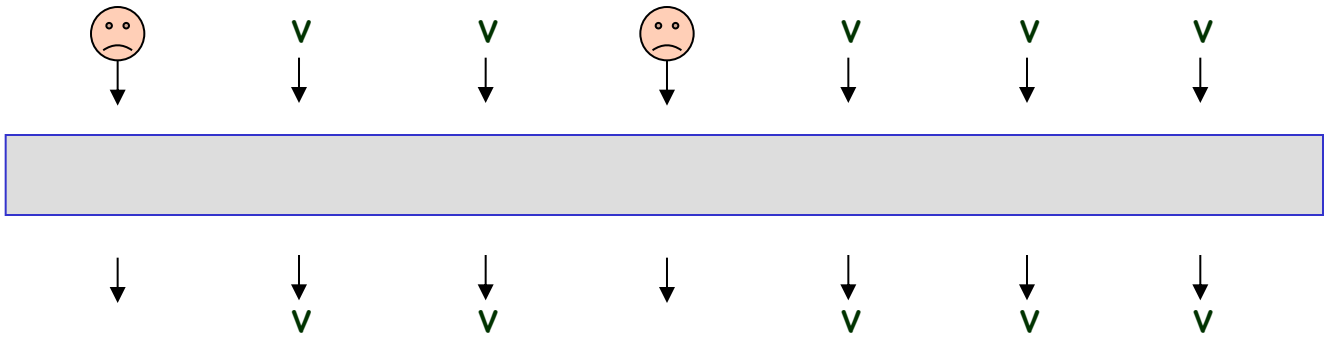
➔ every process got v from > n/4 correct processes

➔ if a process got at least s same messages ➔ must be v

➔ if a process got less than s same messages ➔ choose v

Binary Consensus for Byzantine Failures where n ≥ 4t+1

Lemma 2: same input (of all correct processes) ➔ same output



➔ # of correct processes, s > 3n/4

➔ each correct process gets s messages with the value v

➔ each correct process chooses the value v

time

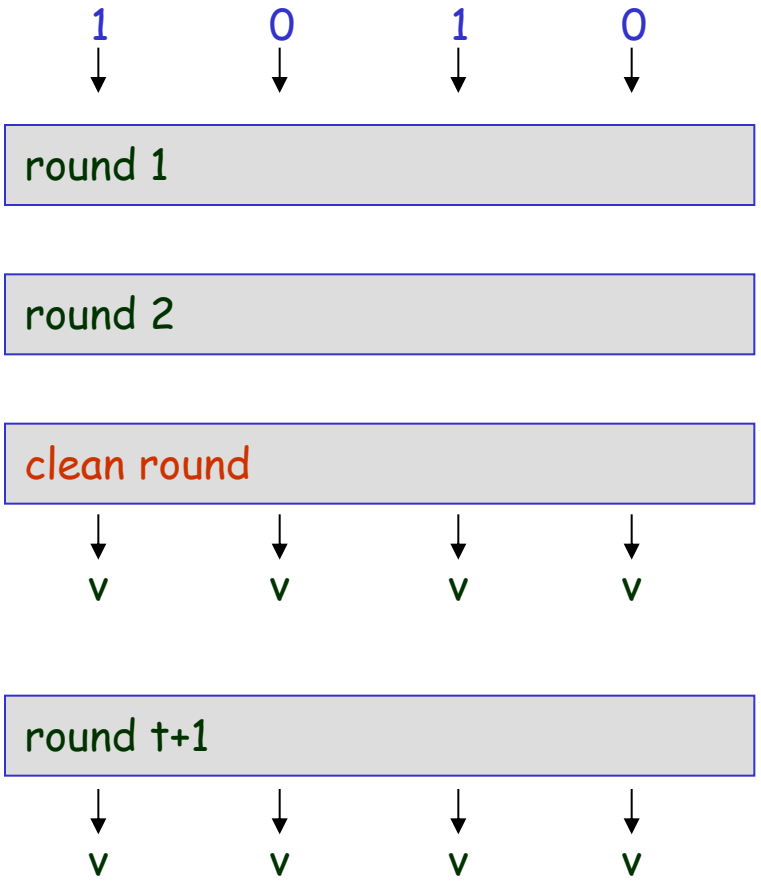
Binary Consensus for Byzantine Failures where n ≥ 4t+1

Lemma 1: clean round ➔ same output

Lemma 2: same input ➔ same output

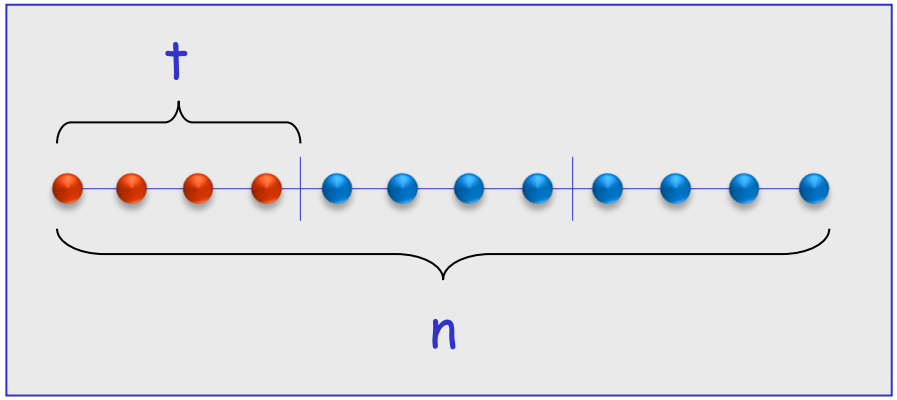
Thm 1: Agreement ✓

Thm 2: Validity ✓



Impossibility of Consensus with n/3 Faulty Byzantine Process

Theorem: There is no synchronous algorithm for n processes that solves the consensus problem using send/receive messages in the presence of n/3 (or more) faulty Byzantine process.



Today

- ✓ DNS:
  - ✓ General description & caching
  - ✓ Actual messages
  - ✓ DNS lookup
  - ✓ DNS privacy
  - ✓ CDN
- ✓ Consensus Algorithms introduction
- ✓ Consensus Algorithm – Crash failures
- ✓ Consensus Algorithm – Byzantine failures